

Delegation Framework Audit Summary

Prepared by Cyfrin

Version 2.0

Lead Auditors

0kage

Al-qaqa

March 18, 2025

Contents

1 About Cyfrin 2

2 About This Document 2

3 Disclaimer 2

4 Copyright and License 2

5 Audit Overview 3

6 Audit Scope 3

7 Risk Classification 5

8 Protocol Summary 5

9 Attestation 5

1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at cyfrin.io.

2 About This Document

This document is a public summary of the security audit Cyfrin performed for Metamask on Delegation Framework. It records the repositories and commits reviewed, the timeline, auditors, in-scope files, and the number and final status of findings by severity. It closes with Cyfrin's attestation of the review's scope and final remediation status.

It is deliberately abbreviated: the technical detail of individual findings is not reproduced here. The complete audit report, version 2.0 dated March 18, 2025, was delivered to Metamask, who retain it as the full record of the engagement. This summary was issued on October 29, 2026.

3 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the in-scope code being audited. Unless the scope explicitly notes otherwise, the review is limited to the application layer and does not extend to the correctness of API/SDK/execution environment implementations, compilers, standard and third-party libraries, underlying operating systems and hardware.

4 Copyright and License

© 2026 Cyfrin Inc. All rights reserved. Material from Metamask or third parties remains the property of its owners.

This summary may be shared only in complete and unmodified form, so that any copy can be checked against the hash in its attestation. Short quotations with attribution to Cyfrin are permitted, provided they do not misrepresent this summary's scope or results. Except as permitted by applicable law, no part of this summary may otherwise be copied, modified, adapted, sold or sublicensed, or used to create derivative works, without Cyfrin's prior written permission.

Cyfrin and the Cyfrin logo are trademarks of Cyfrin Inc. and may not be used to suggest that Cyfrin has reviewed or endorses anything outside this summary's scope, or to misrepresent its results.

5 Audit Overview

Over the course of 11 days, the Cyfrin team conducted an audit on the Delegation Framework code provided by Metamask. The findings consist of 1 High, 2 Medium & 5 Low severity issues with the remainder being informational and gas optimizations.

Summary

Project Name	Delegation Framework
Repository	delegation-framework
Commit	d522a38b0b0f...
Audit Timeline	Feb 12th - Feb 26th, 2025
Methods	Manual Review

Issues Found

Severity	Found	Resolved	Acknowledged
Critical Risk	0	0	0
High Risk	1	1	0
Medium Risk	2	1	1
Low Risk	5	4	1
Informational	4	2	2
Gas Optimizations	1	0	1
Total	13	8	5

6 Audit Scope

The DeleGator Protocol implements an account abstraction layer that enables delegation of permissions across different types of accounts, including EOAs, EIP-7702 accounts, and smart contract accounts. The protocol employs ERC-4337 (Account Abstraction) to manage user operations and authorizations through a flexible delegation framework.

The following contracts were included in the scope of the audit:

Core Protocol

```
EIP7702DeleGatorCore.sol
EIP7702StatelessDeleGator.sol
HybridDeleGator.sol
MultiSigDeleGator.sol
DeleGatorCore.sol
DelegationManager.sol
SimpleFactory.sol
```

Libraries

```
ERC1271Lib.sol
EncoderLib.sol
P256SCLVerifierLib.sol
P256VerifierLib.sol
```

Interfaces

```
ICaveatEnforcer.sol
IDelegationManager.sol
IDeleGatorCore.sol
IERC173.sol
IERC7821.sol
```

Enforcers

```
AllowedCalldataEnforcer.sol
AllowedMethodsEnforcer.sol
AllowedTargetsEnforcer.sol
ArgsEqualityCheckEnforcer.sol
BlockNumberEnforcer.sol
CaveatEnforcer.sol
DeployedEnforcer.sol
ERC20BalanceGteEnforcer.sol
ERC20TransferAmountEnforcer.sol
ERC721BalanceGteEnforcer.sol
ERC721TransferEnforcer.sol
ERC1155BalanceGteEnforcer.sol
IdEnforcer.sol
LimitedCallsEnforcer.sol
NativeBalanceGteEnforcer.sol
NativeTokenPaymentEnforcer.sol
NativeTokenTransferAmountEnforcer.sol
NonceEnforcer.sol
OwnershipTransferEnforcer.sol
RedeemerEnforcer.sol
TimestampEnforcer.sol
ValueLteEnforcer.sol
```

Utilities

```
Constants.sol
Types.sol
```

Scripts

```
DeployEIP7702StatelessDeleGator.s.sol
```

7 Risk Classification

	Impact: High	**Impact: Medium**	**Impact: Low**
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

8 Protocol Summary

DeleGator is an account abstraction protocol designed to enable flexible permission delegation across different types of Ethereum accounts. The protocol allows account owners to delegate specific permissions to other addresses, creating a composable authorization framework that supports complex access control patterns.

Key components of the DeleGator system include:

1. **Smart Contract Accounts (SCAs):** DeleGator implements multiple types of smart contract accounts, including:
   - `HybridDeleGator`: Supports both EOA and P256 (WebAuthn/passkey) signatures
   - `MultiSigDeleGator`: Implements threshold signature schemes with multiple signers
   - `EIP7702StatelessDeleGator`: Integrates with EIP-7702 to enable delegation features on externally owned accounts
2. **Delegation Manager:** Serves as the central verification and execution engine for delegations. It validates delegation chains, enforces access policies, and executes operations on behalf of delegators.
3. **Permission Delegation:** Implements a hierarchical delegation system where permissions can be passed through multiple levels with increasing specificity, allowing for complex authorization patterns.
4. **Caveat Enforcers:** Policy modules that enforce specific constraints on delegations, such as:
   - Time-based restrictions (block number, timestamp)
   - Asset transfer limitations
   - Target address allowlists
   - Method allowlists
   - Payment requirements
5. **ERC-4337 Integration:** Built on the Account Abstraction (ERC-4337) standard, allowing delegations to be executed through user operations without requiring direct interaction from the delegator.
6. **Signature Verification:** Supports multiple signature schemes including ECDSA (for EOAs), P256 (for WebAuthn/passkeys), and multi-signature arrangements.

The protocol aims to solve key challenges in account management and permission delegation, providing a unified and flexible authorization framework for both individuals and organizations operating on Ethereum-based networks.

9 Attestation

Cyfrin attests that it performed a security review of the Delegation Framework code provided by Metamask.

The repositories, commits, review period and findings with their final status are in the Summary and Issues Found tables in Section 5, and the in-scope files in Section 6. This attestation is limited to the documented scope, audited commits and mitigations specifically reviewed by Cyfrin.

Cyfrin publishes this document and its SHA-256 hash at https://github.com/Cyfrin/cyfrin-audit-reports. To check a copy on macOS or Linux, run the following in the folder that contains it; it prints `OK` when the copy matches. If the copy was renamed, set F to its file name. Before running it, check that the URL is Cyfrin's repository.

```
U=https://github.com/Cyfrin/cyfrin-audit-reports/raw/main/reports/sha256
N=2025-03-18-cyfrin-Metamask-DelegationFramework1-v2.0.pdf
F=$N
H=$(curl -fsSL $U/$N.sha256 | cut -d' ' -f1)
echo "$H *$F" | shasum -a 256 -c
```

Cyfrin's full audit report with all findings provided to Metamask has the SHA-256 hash and file name below. To check a copy of it, first check this document as above, then run the following in the folder that contains the full report; it prints `OK` when the copy matches. If the copy was renamed, set F to its file name.

```
H=f79c7d9bd30eb2564d590d7ebd5b0c751a6b9e7f8bc08f3f5e510004edf644c7
F=2025-03-18-cyfrin-Metamask-DelegationFramework1-v2.0-full.pdf
echo "$H *$F" | shasum -a 256 -c
```

On Linux, `sha256sum -c` can be used in place of `shasum -a 256 -c`. On Windows, run `Get-FileHash <file> -Algorithm SHA256` in PowerShell and compare the result with the hash.